

From Flat Stores to Separation Logic: Lessons Building a Python Verifier

Experience report: migrating an LLM-assisted verifier to Iris

Gavin Mendel-Gleason
Scidonia

Abstract

Axiomander is a verification pipeline for ordinary, decorator-free Python: a human writes a strong specification, an LLM proposes the implementation and the helper contracts needed to prove it, and the verifier returns a deterministic verdict with a structured residual on failure. This experience report is about the proof backend underneath that loop. We first built it the obvious way – lowering Python to a small imperative language over a flat store – and recovered the locality that modular verification needs by *reconstructing the frame rule by hand*: an explicit clobber operator and generated per-callee, per-variable frame lemmas. It worked for tiny inputs and then stopped scaling: weakest-precondition terms blew up, the compiled-Ltac automation grew brittle, and whole classes of programs – owned mutable fields, aliasing, anything heading toward concurrency – could not be expressed at all. We then rebuilt the backend on Iris, where a callee’s footprint is a separating resource and framing is the separating conjunction. Our thesis, which the migration bore out, is that the abstractions an LLM-driven verifier needs – modular verification, proof reuse, library stubbing, and incremental checking – are all instances of one mechanism: Iris framing behind opaque specification boundaries. We report what changed quantitatively (framing obligations per call fell to zero; automatically generated, machine-checked proofs replaced hand-decomposed ones) and qualitatively (resource-bearing code became expressible at all), and distil the lessons for others building automated, composition-heavy verifiers.

Keywords

separation logic, Iris, weakest preconditions, specification composition, stubbing, frame rule, LLM-assisted proof, Python, experience report

1 Motivation: vericoding needs composition

Axiomander’s development model is a loop. The human writes a *strong specification* – a precondition/postcondition pair – as the primary artifact. An LLM proposes an implementation. The verifier reduces “does the code meet the spec?” to a proof obligation and returns either a certificate or a *structured residual* (the remaining goal with its hypotheses, plus a counterexample where one exists). The LLM repairs against the residual, not against the original natural-language prompt. Crucially, the LLM has freedom in *sub-specification*: to prove an entry point it may introduce helper functions, each with its own contract, which are checked the same way. The LLM is the reason we need what follows, but it is not the subject of this report: the subject is the proof backend that has to make the loop tractable.

For this loop to be usable on real codebases, three forms of composition must hold simultaneously:

- (1) **Modular verification.** A caller must be provable from a callee’s *contract* alone. If proving f required unfolding the bodies of everything f transitively calls, neither proofs nor LLM prompts would scale.
- (2) **Proof reuse under iteration.** Editing a function body must not invalidate its callers’ proofs as long as the function’s *contract* is unchanged. The verifier should behave like an incremental build system: *body changes invalidate local proofs; contract changes invalidate callers*.
- (3) **Stubbing.** Libraries we cannot or will not verify must be replaceable by their declared behaviour. A caller proves against the stub’s contract, which becomes an assumption; the trust boundary is the stub, written and reviewed once.

All three are, at bottom, the **frame property**: reasoning about one component must be insensitive to the parts of the state it does not touch. The thesis of this report is that these are not three problems but one, that an off-the-shelf separation logic supplies it directly, and that the place we spent our effort – and hit our limits – was trying to reconstruct it over a flat store. The rest of the paper is that story: how we built framing the hard way (Section 2), why we abandoned it for Iris (Section 3), what measurably and expressively changed (Section 4), and what we would tell the next team (Section 6).

2 First architecture: framing by hand over a flat store

Our original backend lowers Python to a small imperative language IMP, with a state that is a flat finite map from variable names to values ($VZ \mid VBool \mid VUnit$, plus structural $VList \mid VTuple \mid VDict$, with mutation modelled on a parallel heap representation). The WP calculus is standard and proven sound in Rocq:

$$\begin{aligned} \text{wp}(\text{skip}, Q) &= Q, \\ \text{wp}(x := e, Q) &= Q[x \mapsto e], \\ \text{wp}(c_1; c_2, Q) &= \text{wp}(c_1, \text{wp}(c_2, Q)), \\ \text{wp}(\text{if } e \text{ } c_1 \text{ } c_2, Q) &= (e \Rightarrow \text{wp}(c_1, Q)) \\ &\quad \wedge (\neg e \Rightarrow \text{wp}(c_2, Q)). \end{aligned}$$

Calls are the interesting case. A call site $t := g(a)$ is verified against g ’s contract using a CCall rule. To recover locality – to let a caller conclude that variables it cares about survive the call – we model the callee’s effect with an explicit clobber operator that havocs the callee’s declared write set, and we must discharge, for

every variable v , an obligation of the form

$$\forall v. v \notin (\text{target} :: \text{writes}) \Rightarrow \\ \text{lget } s \ v = \text{lget } (\text{clobber } (\text{lupd } s \ \text{target } r) \ \text{writes}) \ v.$$

This is the frame rule, but encoded extensionally over a flat store. Three problems follow, all of which we hit in practice. Two are about cost:

- **WP-term blowup.** Each successive call nests the full WP expansion inside the previous call’s postcondition. A function with a handful of sequential calls produces terms too large for the kernel to handle comfortably, and the monolithic proof tactic stops closing them.
- **Brittle automation.** The single universally-quantified frame subgoal does not match cleanly in compiled Ltac, partly because coercions (e.g. an `ls` wrapper) normalise away in the `.vo` and defeat the source-level pattern.

The third is about *expressivity*, and it is the one we underestimated: a flat name-to-value store cannot model two object references that alias the same cell, cannot transfer ownership of a mutable field across a call, and has no path to concurrency. Whole programs – not just whole proofs – fall outside it.

Our mitigation for the cost problems was to generate, at the Python IR level, **one frame lemma per (callee, preserved-variable) pair**:

```
(* g writes ["result"]; the caller's "a" is
   preserved across t := g(a) *)
Lemma g_frame_a : forall (s : state) (r : Z),
  ~ In "a" ("t" :: "result" :: nil) ->
  lget s "a" =
    lget (clobber (lupd s "t" (VZ r))
           ("result" :: nil)) "a".
Proof. apply wp_ccall_frame. Qed.
```

The caller’s proof then replaces the monolithic $\forall v$ subgoal with a sequence of `apply g_frame_a. apply g_frame_b. ...`, each trivial and each failing independently with its own residual. This restores tractability and keeps the per-obligation, fail-locally discipline we want. But it is, frankly, a workaround: we are *reconstructing* the frame rule, one variable at a time, in a logic that does not have it – and reconstruction does nothing for the expressivity gap. The lesson is the familiar one: **if framing is load-bearing, the assertion logic should be substructural**.

3 Migration to Iris

We rebuilt the backend on Iris. **Snakelet** is the intermediate language into which Axiomander lowers Python; **SnakeletWp** is its weakest-precondition calculus, defined inside the Iris program logic. The state is an Iris heap: a mutable Python object field is a points-to assertion $l \mapsto v$, ownership is a separating resource rather than an entry in a flat map, and assertions are `iProp`. Snakelet targets Python *in the large* – ordinary control flow, exceptions, calls, and the built-in containers lower directly, and only genuinely pathological constructs (runtime metaprogramming, reflection) are gated with a diagnostic. Its full term language, small-step semantics, and the machine-checked WP rule set are in Appendix A; this section is about what the move bought us.

Table 1: Contract fragments compiled to `iProp`.

Python contract fragment	<code>iProp</code>
<code>assert x >= 1</code>	$\ulcorner x \geq 1 \urcorner$
<code>assert implies(A, B)</code>	$\ulcorner A \rightarrow B \urcorner$
<code>assert all(x != i for i in range(0,10))</code>	$\ulcorner \forall i, 0 \leq i < 10 \rightarrow x \neq i \urcorner$
<code>owns(box)</code>	$\text{box.value} \mapsto v \text{ (a resource)}$

3.1 Contracts compile to `iProp`, composed with the separating conjunction

Contracts remain ordinary Python – plain `assert` statements (leading = precondition, trailing-before-return = postcondition, in-loop = invariant) or a verifier-only axiomander: `docstring`. A shared `ContractLinter` parses them into a contract IR; a backend-specific pass (`contract_ir_iris`) compiles that IR to `iProp`. Pure facts are injected with the persistent embedding $\ulcorner P \urcorner$; resource facts compile to points-to assertions.

The decisive design choice is that **the default composition of contract clauses is the separating conjunction $*$, not $\wedge$** . When a precondition or postcondition is built from several `assert`s, they are joined with $*$. For the pure-integer fragment this costs nothing, since $\ulcorner P \urcorner * \ulcorner Q \urcorner \dashv\vdash \ulcorner P \wedge Q \urcorner$; but it means the *same* composition rule already accounts for disjoint resources, and it is the connective that lets us scale to invariants and atomic updates without changing the front end.

A typical mixed pure/resource contract, written by the user with no Iris notation:

```
def bump(box: Box) -> int:
    """
    axiomander:
        requires:
            owns(box)
            box.value >= 0
        modifies:
            box.value
        ensures:
            owns(box)
            box.value == old(box.value) + 1
            result == box.value
    """
    box.value = box.value + 1
    return box.value
```

compiles to the Hoare triple

$$\{ \text{box.value} \mapsto n * \ulcorner n \geq 0 \urcorner \} \quad \text{bump}(\text{box}) \\ \{ r. \text{box.value} \mapsto n+1 * \ulcorner r = n+1 \urcorner \}.$$

`owns(box)` plus `modifies: box.value` is the resource footprint; the user never writes $*$, $\mapsto$, masks, or proof-mode tactics. Note that this contract has no flat-store rendering at all: `owns` denotes ownership of a mutable cell, which the name-to-value store of Section 2 cannot express. Resource awareness is the first thing the migration *adds*, before any question of proof size.

3.2 The frame rule is free, and so is composition

Because assertions are `iProp`, the structural frame rule

$$\frac{\{P\} c \{Q\}}{\{P * R\} c \{Q * R\}}$$

holds without a side condition: any resource R describing a disjoint region of the heap is carried across c by the separating conjunction, automatically. Compared with Section 2, the entire clobber/ $\forall v \notin \text{writes}$ apparatus, and the per-callee per-variable frame-lemma generator that papered over it, simply disappear. Locality stops being something we encode and becomes the shape of the logic.

3.3 Opaque calls: one boundary, three jobs

Calls are dispatched through a **contract table** (an Iris FunCtx) that maps each callee to its specification. SnakeletWp provides two call rules: a *transparent* call unfolds a first-party body, and an *opaque* call consumes the callee’s precondition (giving up the relevant resources), receives its postcondition, and frames the rest. Schematically, for a callee with spec $\{P_g\} g \{Q_g\}$:

$$\frac{P_g * R \quad \forall r. (Q_g * R) \vdash \text{wp } K \{ \Phi \}}{\text{wp } (\text{call } g; K) \{ \Phi \}}$$

The single most useful thing we learned is that this one rule does three jobs at once. It is **stubbing**: a library function given a contract – in a .pyi stub or an axiomander: docstring – enters the FunCtx and callers prove against it, body never required. It is **modular verification**: a first-party callee can be re-declared opaque once its contract is stable. And it is the **incremental-reuse boundary**: callers depend only on the contract, so a body edit cannot disturb them, and the FunCtx entry’s hash is exactly the key our cache uses to decide what to re-prove. Modularity, stubbing, and incrementality are not three features we built; they are three readings of the opaque-call rule.

3.4 Separation logic as an implementation detail

The migration is deliberately confined to the proof backend. The ContractLinter, the SMT export (smt_export, theory_smt), the LLM oracle, and the function-level incremental cache are all backend-agnostic; the contract IR diverges only at the final rendering step (flat-store to_coq vs. contract_ir_iris.iris_prop). From the user’s side nothing changes: contracts are still decorator-free Python, and owns(...) is the only new, entirely optional, vocabulary. The separating conjunction, the points-to, the magic wand, and the FunCtx never surface. Iris is the substrate, not the interface – separation logic as an *implementation detail*.

3.5 Staged, syntax-directed proofs

SnakeletWp ships a small instruction set of **stage tactics** – pure_step, case_bool, call_opaque, call_transparent, finish_pure, and a precondition-discharge stage call_opaque_pre – and the proof generator (iris_proof_gen) emits exactly one stage per IR node, chosen by the node’s syntax and the callee’s table entry. The generator performs no symbolic execution: each stage tactic extracts what it needs from the Iris goal at proof time, so a three-call chain becomes a flat, machine-checked sequence such as [call_opaque, pure_step, call_transparent, pure_step, pure_step, call_opaque_pre, finish_pure]. Crucially for the LLM loop, **stages fail independently**: when call_opaque_pre cannot discharge a precondition the residual is exported to SMT (an UNSAT result returns as Axiom smt_ax_N, the stage regenerated as call_opaque_pre (exact

Table 2: Framing cost and proof size on functions from the repository. *framing* counts explicit per-call frame steps plus preservation conjuncts threaded through intermediate postconditions (“2+14” = 2 frame applications + 14 conjuncts); *proof* is non-blank Proof-Qed lines. Flat-store proofs are hand-decomposed (the monolithic tactic does not scale past one call); Iris proofs are generated and coqc-checked. “n/a” = not expressible over a flat store; “–” = no scalable proof.

function (calls)	flat store		Iris	
	framing	proof	framing	proof
2-call chain	2+14	29	0	11
3-call chain	–	–	0	14
owned-field bump	n/a	n/a	0	15
mutate-then-raise	n/a	n/a	0	9

smt_ax_N)); a pure postcondition finish_pure cannot close is likewise an SMT candidate; and beyond SMT the named stage plus its goal state – not a stderr regex – is what the LLM oracle receives. A failed attempt yields a reusable artifact rather than a dead end.

4 Evaluation

We compare the two backends on small functions drawn from the repository. Two effects matter, and the second is the one we care about more: how much framing the proof must do by hand, and what can be expressed at all.

Framing cost. The flat-store backend pays for locality per call: at each call site it must establish that every live variable outside the callee’s write set is preserved. For the two-call function frame_two_calls (two increments and a sum) this means two explicit wp_ccall_frame applications and fourteen preservation conjuncts threaded through intermediate postconditions (Q_1 : 6, Q_2 : 8); the monolithic tactic does not close it, so the proof is a hand-written staged decomposition. In Iris the frame rule carries untouched resources implicitly, so the corresponding obligation count is *zero* – independent of how many variables are live or how many calls precede. Table 2 reports this against generator-emitted, coqc-checked SnakeletWp proofs.

The trend, not the absolute numbers, is the point: flat-store framing grows with (calls $\times$ live variables) and is discharged by hand, while the Iris column is a flat zero and the proofs are emitted automatically. The three-call Iris proof (14 lines) is *shorter* than the two-call flat-store proof (29 lines) despite doing more.

Expressivity, not just efficiency. The bottom two rows are the more important result. bump mutates an owned field; mutate-then-raise mutates a cell and then raises, with the exceptional postcondition describing the heap as it stood at the raise. Neither has a flat-store analogue at all – the name-to-value store cannot express ownership of a mutable cell, let alone ownership surviving an exception. The migration did not merely make these proofs shorter; it made the corresponding *programs verifiable for the first time*. The same observation extends to aliasing and to the concurrency primitives the calculus already carries: separation logic widened what Axiomander can talk about, and proof size is a secondary benefit.

5 Discussion and related work

Axiomander’s separation-logic interpretation of object fields and its opaque call rule are textbook Iris; the novelty is not in the logic but in the *system context*, and the contribution of this report is the experience of the migration rather than a new metatheory. To our knowledge the combination is unusual: the assertion logic is never exposed to the user, the proof obligations are generated and largely discharged by an LLM operating on structured residual goals with Iris/Rocq as the trust kernel, and a single opaque-call rule serves as modular verification, incremental-reuse boundary, and library stub at once. We offer Axiomander as a data point that Iris’s framing and ownership story is the right substrate for *automated*, composition-heavy verification, not only for expert-driven interactive proof.

6 Lessons learned

- (1) **Framing is the fundamental abstraction.** Every composition property we needed reduced to it. Building it by hand over a flat store is where the WP blowup and the brittle automation came from; the substructural logic makes the cost vanish.
- (2) **One boundary does many jobs.** Opaque specifications unify modular verification, proof reuse, and library stubbing – and the incremental cache keys on the same contract hash. Designing them as one mechanism, rather than three, simplified the whole pipeline.
- (3) **Incrementality follows contract boundaries.** Once callers depend only on contracts, “what must I re-prove after this edit?” has a syntactic answer (the changed contract hashes), and per-stage hashing refines it further.
- (4) **Ownership is an expressivity question, not only efficiency.** Real Python mutates shared objects and raises through them. A flat store cannot express that, so a substructural logic is what makes such code verifiable at all – the smaller proofs are a bonus.
- (5) **Separation logic is easier to adopt than to reconstruct.** We spent more engineering faking the frame rule – clobber operators, generated lemmas, cbn exclusions – than we did inheriting it from Iris. If framing is load-bearing, reach for the substructural logic first.

Status. Snakelet and SnakeletWp are complete and free of Admitted across the rule set of Appendix A, including the exception-aware extension and its loop rules; the staged generator, the end-to-end Python-to-Iris pipeline, the contract-to-iProp compiler, and the SMT-axiom slot are in place, with tens of Iris-specific tests passing alongside the mature flat-store backend. Remaining work wires the exception calculus into the Python lowering and extends the fragment to loop invariants over shared disjoint memory, in-place data-structure mutation, and type-aware reasoning over strings/floats/isinstance/None.

Open questions. Two we would most value input on: how far an object’s footprint can be *inferred* from modifies for an LLM-generated body before the user must annotate owns(...); and which Iris proof-mode presentation of a failed stage’s residual (hypothesis naming, spatial/persistent split, masks elided) makes the best LLM prompt.

```

v ::= n | b | fl | s | l | () | exn s v
    | [  $\bar{v}$  ] | (  $\bar{v}$  ) | {  $\bar{v}$ ;  $\bar{v}$  } | {  $\bar{v}$  }
op ::= + | - |  $\times$  | / | % | = |  $\neq$  | < |  $\leq$  | > |  $\geq$ 
    | and | or | in | len | union | inter
e ::= v | x | let x=e in e | op(e, e) | g( $\bar{e}$ )
    | !e | e := e | ref e | if e then e else e
    | raise e | try e catch x $\Rightarrow$ e
    | while e e | for x in e do e
K ::= let x= $\square$  in e | op( $\square$ , v) | op(e,  $\square$ ) | ! $\square$  | ref  $\square$ 
    |  $\square$  := v | e :=  $\square$  | if  $\square$  then e else e | raise  $\square$ 
    | try  $\square$  catch x $\Rightarrow$ e | for x in  $\square$  do e
r ::= val v | exn s v

```

Figure 1: Snakelet term language: values v (integer, boolean, float, string, location, unit, exception object, and the structural list/tuple/dict/set), operators op , expressions e , single-item evaluation contexts K , and results r . Every K is *neutral* except try $\square \dots$; $op(e, e)$ evaluates its right operand first. The core development additionally provides FAA, fork, and dictionary get/set.

Artifact pointers

This paper summarises material developed in the Axiomander repository: the Iris migration plan and current state; the Iris backend prototype (the bump/owns example and resource classification); the per-callee frame lemmas in the flat-store backend (Section 2); the staged proof engineering notes; the incremental-verification cache design; and the companion whitepaper and slides motivating vericoding.

A The Snakelet calculus

Snakelet is an expression language in administrative-normal form: every operator and call takes values or variables, and evaluation order is made explicit by let-binding and a per-item evaluation-context grammar. A store is a finite map from locations to values ($\sigma : \text{loc} \rightarrow v$); a mutable Python object field is a heap cell, exposed in the logic as $l \mapsto v$. The immutable Python containers – list, tuple, dict, set – are *structural* values, not heap objects, mirroring Dafny’s seq and F*’s list.

Two coordinated developments share this design. The core language carries float literals, dictionary get/set, and the concurrency primitives FAA/fork; the exception-aware extension shown here adds first-class exceptions and carries the complete, machine-checked rule set. We present the latter because exceptions change the *shape of the judgement*. Following van Collem/de Vilhena/Krebbers (PLDI 2026), raise v on a value is a **stuck terminal** expression – not itself a value – so an uncaught raise sits in evaluation position as the program’s exceptional result. A program terminates in one of two ways, captured by the result type $r ::= \text{val } v \mid \text{exn } s \ v$, and the WP postcondition ranges over r rather than over values. Figure 1 gives the term language.

Operational semantics. Reduction is small-step and factors into *pure* steps $e \leadsto e'$ (no heap, deterministic) and *heap* steps $(e, \sigma) \rightarrow (e', \sigma'; ef)$ (heap and calls), each lifted under an evaluation context by CTX-P/CTX-H (Figure 2). All context items are neutral except try $\square$ catch x $\Rightarrow$ h; the unwinding rule fires only for neutral K , so a raise propagates through let, operators, and loads but is caught by

an enclosing `try`. Calls dispatch through the contract table entries: `FunSpec P Q` (opaque – stuck unless P holds) or `FunDef  $\vec{x}$  b` (transparent – substitute into the body).

Weakest preconditions. SnakeletWp’s postcondition is $\Phi : \text{Result} \rightarrow iProp$. It is the guarded fixpoint of: if e is terminal then Φr holds under a fancy update; otherwise e is reducible and, after one step, the state interpretation (the Iris `gen_heap` authoritative view) and the WP of the reduct are re-established. Ranging Φ over `Result` is the one structural change exceptions force. Composition goes through `BIND`, which threads a `Result`-typed continuation `bindp K  $\Phi$` with `bindp K  $\Phi$  (val  $v$ ) = WPE K[ $v$ ]  $\{\{\Phi\}\}$` and `bindp K  $\Phi$  (exn  $s v$ ) =  $\Phi(\text{exn } s v)$` – on a value it continues, on an exception it short-circuits

to the exceptional postcondition. The heap rules are the usual Iris points-to rules; their exceptional arm describes the heap *as it stood at the raise*. On top of `FORCONS` we derive a structural fold rule carrying a user invariant P over the remaining list suffix:

$$PM * (\Box \forall v \text{ vs. } P(v :: \text{vs}) \multimap \text{WPE } b[x:=v] \{\{r. \theta(r)\}\}) \\ * (P [] \multimap \Phi(\text{val } ())) \vdash \text{WPE for } x \text{ in } M \text{ do } b \{\{\Phi\}\},$$

where $\theta(\text{val } _) = P \text{ vs}$ and $\theta(\text{exn } s v) = \Phi(\text{exn } s v)$. Each rule has a matching stage tactic (Section 3.5): `pure_step` fires `LET/OP/IFT/IFF/TRYV/TRYC`, `heap_load/store/alloc` the heap rules, `call_opaque/call_transparent` the call rules, `raise_step` `RAISE/UNWIND`, and `loop_unfold` the loop rules.

$$\begin{array}{c}
\textbf{Pure reduction} \quad e \rightsquigarrow e' \\
\\
\text{LET} \frac{}{\text{let } x=v \text{ in } e \rightsquigarrow e[x:=v]} \quad \text{OP} \frac{}{op(v_1, v_2) \rightsquigarrow \text{binop}(op, v_1, v_2)} \quad \text{IFT} \frac{}{\text{if true then } e_1 \text{ else } e_2 \rightsquigarrow e_1} \quad \text{IFF} \frac{}{\text{if false then } e_1 \text{ else } e_2 \rightsquigarrow e_2} \\
\\
\text{TRYV} \frac{}{\text{try } v \text{ catch } x \Rightarrow h \rightsquigarrow v} \quad \text{TRYC} \frac{}{\text{try (raise } v) \text{ catch } x \Rightarrow h \rightsquigarrow h[x:=v]} \quad \text{UNWIND} \frac{\text{neutral}(K)}{K[\text{raise } v] \rightsquigarrow \text{raise } v} \\
\\
\text{WHILE} \frac{}{\text{while } e_1 \ e_2 \rightsquigarrow \text{if } e_1 \text{ then } (e_2 ; \text{while } e_1 \ e_2) \text{ else } ()} \quad \text{FORNIL} \frac{}{\text{for } x \text{ in } [] \text{ do } b \rightsquigarrow ()} \\
\\
\text{FORCONS} \frac{}{\text{for } x \text{ in } (v :: vs) \text{ do } b \rightsquigarrow b[x:=v] ; \text{for } x \text{ in } vs \text{ do } b} \\
\\
\textbf{Head reduction} \quad (e, \sigma) \rightarrow (e', \sigma'; \vec{ef}) \quad \text{and} \quad \textbf{context decomposition} \\
\\
\text{LOAD} \frac{\sigma(l) = v}{(!l, \sigma) \rightarrow (v, \sigma)} \quad \text{STORE} \frac{l \in \text{dom } \sigma}{(l := v, \sigma) \rightarrow ((), \sigma[l \mapsto v])} \quad \text{ALLOC} \frac{\sigma(l) = \perp}{(\text{ref } v, \sigma) \rightarrow (l, \sigma[l \mapsto v])} \\
\\
\text{CALLSPEC} \frac{\text{entries}(g) = \text{FunSpec } P \ Q \quad P \ \vec{v} \quad Q \ \vec{v} \ v}{(g(\vec{v}), \sigma) \rightarrow (v, \sigma)} \quad \text{CALLDEF} \frac{\text{entries}(g) = \text{FunDef } \vec{x} \ b \quad |\vec{v}| = |\vec{x}|}{(g(\vec{v}), \sigma) \rightarrow (b[\vec{x} := \vec{v}], \sigma)} \quad \text{CTX-P} \frac{e \rightsquigarrow e'}{(K[e], \sigma) \rightarrow (K[e'], \sigma)} \\
\\
\text{CTX-H} \frac{(e, \sigma) \rightarrow (e', \sigma'; \vec{ef})}{(K[e], \sigma) \rightarrow (K[e'], \sigma'; \vec{ef})}
\end{array}$$

Figure 2: Snakelet operational semantics. Pure steps are heap-independent and deterministic; head steps touch the heap or the contract table entries. **CALLSPEC** fires only inside the callee’s precondition P (calling outside P is stuck), making the precondition a proof obligation on the caller. $\vec{ef}$ collects forked threads (empty except for fork in the core development).

$$\begin{array}{c}
 \text{VAL} \frac{}{\Phi(\text{val } v) \vdash \text{WPE } v \llbracket \Phi \rrbracket} \quad \text{RAISE} \frac{}{\Phi(\text{exn } s \ v) \vdash \text{WPE raise } (\text{exn } s \ v) \llbracket \Phi \rrbracket} \quad \text{WAND} \frac{}{\text{WPE } e \llbracket \Phi \rrbracket * (\forall r. \Phi \ r \multimap \Psi \ r) \vdash \text{WPE } e \llbracket \Psi \rrbracket} \\
 \\
 \text{LET} \frac{}{\triangleright \text{WPE } e[x:=v] \llbracket \Phi \rrbracket \vdash \text{WPE let } x=v \text{ in } e \llbracket \Phi \rrbracket} \quad \text{LETRAISE} \frac{}{\Phi(\text{exn } s \ v) \vdash \text{WPE let } x=\text{raise } (\text{exn } s \ v) \text{ in } e \llbracket \Phi \rrbracket} \\
 \\
 \text{OP} \frac{}{\triangleright \text{WPE binop}(op, v_1, v_2) \llbracket \Phi \rrbracket \vdash \text{WPE } op(v_1, v_2) \llbracket \Phi \rrbracket} \quad \text{IFT} \frac{}{\triangleright \text{WPE } e_1 \llbracket \Phi \rrbracket \vdash \text{WPE if true then } e_1 \text{ else } e_2 \llbracket \Phi \rrbracket} \\
 \\
 \text{IFF} \frac{}{\triangleright \text{WPE } e_2 \llbracket \Phi \rrbracket \vdash \text{WPE if false then } e_1 \text{ else } e_2 \llbracket \Phi \rrbracket} \quad \text{TRYV} \frac{}{\triangleright \text{WPE } v \llbracket \Phi \rrbracket \vdash \text{WPE try } v \text{ catch } x \Rightarrow h \llbracket \Phi \rrbracket} \\
 \\
 \text{TRYC} \frac{}{\triangleright \text{WPE } h[x:=v] \llbracket \Phi \rrbracket \vdash \text{WPE try } (\text{raise } v) \text{ catch } x \Rightarrow h \llbracket \Phi \rrbracket} \quad \text{BIND} \frac{\text{neutral}(K)}{\text{WPE } e \llbracket \text{bindp } K \Phi \rrbracket \vdash \text{WPE } K[e] \llbracket \Phi \rrbracket} \\
 \\
 \text{LOAD} \frac{}{l \mapsto v * \triangleright (l \mapsto v \multimap \Phi(\text{val } v)) \vdash \text{WPE } !l \llbracket \Phi \rrbracket} \quad \text{STORE} \frac{}{l \mapsto w * \triangleright (l \mapsto v \multimap \Phi(\text{val } ())) \vdash \text{WPE } l := v \llbracket \Phi \rrbracket} \\
 \\
 \text{ALLOC} \frac{}{\triangleright (\forall l. l \mapsto v \multimap \Phi(\text{val } l)) \vdash \text{WPE ref } v \llbracket \Phi \rrbracket} \quad \text{FORNIL} \frac{}{\triangleright \Phi(\text{val } ()) \vdash \text{WPE for } x \text{ in } [] \text{ do } b \llbracket \Phi \rrbracket} \\
 \\
 \text{CALLOPAQUE} \frac{\text{entries}(g) = \text{FunSpec } P \ Q \quad P \vec{v}}{\triangleright (\forall v. \ulcorner Q \vec{v} v^{-1} \multimap \Phi(\text{val } v) \urcorner \vdash \text{WPE } g(\vec{v}) \llbracket \Phi \rrbracket)} \quad \text{CALLDEF} \frac{\text{entries}(g) = \text{FunDef } \vec{x} \ b \quad |\vec{v}| = |\vec{x}|}{\triangleright \text{WPE } b[\vec{x}:=\vec{v}] \llbracket \Phi \rrbracket \vdash \text{WPE } g(\vec{v}) \llbracket \Phi \rrbracket} \\
 \\
 \text{WHILE} \frac{}{\triangleright \text{WPE if } e_1 \text{ then } (e_2 ; \text{while } e_1 \ e_2) \text{ else } () \llbracket \Phi \rrbracket} \\
 \vdash \text{WPE while } e_1 \ e_2 \llbracket \Phi \rrbracket \\
 \\
 \text{FORCONS} \frac{}{\triangleright \text{WPE } b[x:=v] ; \text{for } x \text{ in } vs \text{ do } b \llbracket \Phi \rrbracket} \\
 \vdash \text{WPE for } x \text{ in } (v :: vs) \text{ do } b \llbracket \Phi \rrbracket
 \end{array}$$

Figure 3: SnakeletWp weakest-precondition rules. The postcondition $\Phi : \text{Result} \rightarrow iProp$ ranges over $r ::= \text{val } v \mid \text{exn } s \ v$; $\triangleright$ is the Iris later modality, $*$ the separating conjunction, $\multimap$ the magic wand, and $\ulcorner \cdot \urcorner$ the persistent embedding of a pure proposition. **BIND** uses the Result-transformer `bindp` (text); the wider **WHILE**/**FORCONS** rules are shown beneath the grid. Every rule is machine-checked.